



Luffy Challenge

Montar el entorno del desafío

Primero, debemos iniciar el entorno para completar el desafío para ello es necesario levantar el contenedor de docker utilizando el primer comando documentado en  [Utils](#).

Luego, debemos cambiar de usuario al correspondiente al reto, para ello utilizamos el segundo comando documentado en  [Utils](#).

```
PS C:\Users\Usuario\Documents\GitHub\data-encryption\ctf_onepice_symmetric_cipher> docker exec -it luffy_challenge bash
nobody@6d07a1328137:/home/luffy$ su luffy
Password:
luffy@6d07a1328137:~$
```

La contraseña es `onepiece`

Extraer bandera de `flag.txt`

El primer paso para extraer la bandera del desafío, es encontrar todos los posibles archivos que puedan contenerla, para ello, utilizamos el tercer comando documentado en  [Utils](#).

```
luffy@6d07a1328137:~/ONEPIECE$ find . -type f -iname "*flag*"
./East_Blue/Loguetown/Casa_de_Bell-m??re/flag.txt
./East_Blue/Loguetown/Casa_de_Smoker/flag.txt
./East_Blue/Shells_Town/Casa_de_Morgan/flag.txt
./East_Blue/Baratie/Casa_de_Zeff/flag.txt
./Alabasta/Alubarna/Casa_de_Crocodile/flag.txt
./Alabasta/Alubarna/Casa_de_Vivi/flag.txt
./Water_7/Carpenters_Cafe/Casa_de_Iceburg/flag.txt
./Water_7/Carpenters_Cafe/Casa_de_Lucci/flag.txt
./Water_7/GalleyLa_Headquarters/Casa_de_Lucci/flag.txt
./Pirate_Island/Pirates_Tavern/Casa_de_Gold_Roger/flag.txt
./Pirate_Island/Pirates_Hideout/Casa_de_Whitebeard/flag.txt
luffy@6d07a1328137:~/ONEPIECE$
```

Luego de obtener todos los potenciales archivos, verificamos el contenido de cada uno, utilizando el comando `cat`, hasta encontrar el archivo que contiene texto cifrado.

```
luffy@6d07a1328137:~$ cat ./ONEPIECE/Alabasta/Alubarna/Casa_de_Crocodile/flag.txt
747d74736d560307050002540257565757030c540b030d03510a50530d0a0b505100070609luffy@6d07a1328137:~$
luffy@6d07a1328137:~$
```

Para descifrar la bandera, utilizamos un script que realiza una operación XOR entre el texto cifrado y mi carnet de estudiante (21542).

```
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Uso: $0 <ruta_del_archivo> <clave_xor>"
    exit 1
fi

FILE_PATH=$1
XOR_KEY=$2

HEX_CONTENT=$(cat "$FILE_PATH")

echo "$HEX_CONTENT" | xxd -r -p | python3 -c "
```

```
import sys
key='$XOR_KEY'
data=sys.stdin.buffer.read()
print(''.join(chr(data[i]^ord(key[i%len(key)]))) for i in range(len(data))))
"
```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```
luffy@6d07a1328137:~/ONEPIECE$ cat ./Alabasta/Alubarna/Casa_de_Crocodile/flag.txt
747d74736d560307050002540257565757030c540b030d03510a50530d0a0b505100070609luffy@6d07a1328137:~/ONEPIECE$
luffy@6d07a1328137:~/ONEPIECE$ nano decrypt.sh
luffy@6d07a1328137:~/ONEPIECE$ chmod +x decrypt.sh
luffy@6d07a1328137:~/ONEPIECE$ ./decrypt.sh ./Alabasta/Alubarna/Casa_de_Crocodile/flag.txt "21542"
FLAG_d22120e7cdef68f9287c8af989ad4548
luffy@6d07a1328137:~/ONEPIECE$
```

Obteniendo así, la bandera descriptada.

```
FLAG_d22120e7cdef68f9287c8af989ad4548
```

Extraer párrafo de poneglyph.jpeg

Al igual que con la extracción de la bandera, el primer paso es encontrar la carpeta que contiene la imagen cifrada, para ello, utilizamos el cuarto comando documentado en  [Utils](#).

```
luffy@6d07a1328137:~/ONEPIECE$ find . -type f -name "*poneglyph*"
./Alabasta/Spiders_Cafe/Casa_de_Mr. 0/poneglyph.zip
luffy@6d07a1328137:~/ONEPIECE$ cd "./Alabasta/Spiders_Cafe/Casa_de_Mr. 0/"
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr. 0$
```

Luego de obtener la carpeta, nos dirigimos hacia su directorio y la extraemos su contenido utilizando el quinto comando documentado en  [Utils](#). La

contraseña a utilizar es la misma que la del usuario.

```
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr. 0$ 7z x poneglyph.zip -oponeglyph
7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=C,Utf16=off,HugeFiles=on,64 bits,8 CPUs Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz (806EA),ASM,AES-NI)

Scanning the drive for archives:
1 file, 69937 bytes (69 KiB)

Extracting archive: poneglyph.zip
--
Path = poneglyph.zip
Type = zip
Physical Size = 69937

Enter password (will not be echoed):
Everything is Ok

Size:          71375
Compressed:    69937
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr. 0$ ls
poneglyph  poneglyph.zip
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr. 0$
```

Para poder utilizar el comando de extracción, es posible que sea necesario instalar ciertas librerías. Para ello, utilice el noveno comando documentado en  [Utils](#).

Para descifrar el párrafo, utilizamos un script que:

1. Extrae el texto de la imagen.
2. Toma el texto de la imagen y realiza una operación XOR entre el texto cifrado y mi carnet de estudiante (21542).

```
#!/bin/bash

#Verificar que se proporcione el carné como argumento
if [ $# -ne 1 ]; then
    echo "Uso: $0 <tu_carné>"
    exit 1
fi

STUDENT_ID=$1

#Definir los nombres de los directorios y archivos
IMAGE_PATH="poneglyph.jpeg" # Asumiendo que está en el mismo directorio
OUTPUT_FILE="decrypted.txt" # Archivo donde se guardará el texto descifrado
```

```

#Crear el archivo luffy_xor.py con el contenido correcto
cat > luffy_xor.py << 'EOF'
def xor_cipher(text, key):
    # Convertir tanto el texto como la clave a bytes
    if type(text) == str:
        text_bytes = text.encode()
    else:
        text_bytes = text
    key_bytes = key.encode() # Convertir la clave a bytes

    # Cifrar con XOR
    cipher_text = bytes([text_bytes[i] ^ key_bytes[i % len(key_bytes)] for i in range(len(text_bytes))])

    return cipher_text
EOF

#Crear un script Python temporal para la extracción
cat > extract_script.py << 'EOF'
from PIL import Image
import piexif
import sys
from luffy_xor import xor_cipher

def extraer_texto_metadata(imagen_path):
    # Abrir la imagen
    img = Image.open(imagen_path)

    # Obtener los metadatos EXIF
    try:
        exif_data = img.info.get('exif')
        if not exif_data:
            return None

        exif_dict = piexif.load(exif_data)

        # Obtener el texto almacenado en 'Artist'
        texto = exif_dict['0th'].get(piexif.ImageIFD.Artist)

```

```

        if texto:
            return texto
        return None
    except Exception as e:
        print(f"Error al extraer metadatos: {e}")
        return None

# Obtener argumentos
if len(sys.argv) != 4:
    print("Uso: python script.py <imagen_path> <carne> <output_file>")
    sys.exit(1)

imagen_path = sys.argv[1]
student_id = sys.argv[2]
output_file = sys.argv[3]

# Extraer y descifrar
texto_cifrado = extraer_texto_metadata(imagen_path)
if texto_cifrado:
    try:
        # Aplicar XOR y decodificar si es posible
        texto_descifrado = xor_cipher(texto_cifrado, student_id)
        try:
            # Intentar decodificar como UTF-8
            decoded_text = texto_descifrado.decode('utf-8')

            # Guardar en archivo
            with open(output_file, 'w') as f:
                f.write(decoded_text)

            # Mostrar en pantalla
            print(decoded_text)

        except UnicodeDecodeError:
            # Si falla la decodificación UTF-8, guardar como bytes en archivo bina
            with open(output_file, 'wb') as f:
                f.write(texto_descifrado)
    
```

```

# Mostrar en pantalla
print(texto_descifrado)
print(f"\nEl contenido binario descifrado se ha guardado en: {output_file}")

except Exception as e:
    print(f"Error al descifrar: {e}")
else:
    print("No se encontraron metadatos EXIF o no hay campo Artist en la imagen")
EOF

#Ejecutar el script en el mismo contenedor Docker
pip install pillow piexif

python3 extract_script.py "$IMAGE_PATH" "$STUDENT_ID" "$OUTPUT_FILE"

echo "Texto descifrado guardado en: $OUTPUT_FILE"

rm -f luffy_xor.py extract_script.py

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```

luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ ls
poneglyph.jpeg
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ nano decryptImg.sh
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ chmod +x decryptImg.sh
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ ./decryptImg.sh "21542"
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pillow in /home/luffy/.local/lib/python3.10/site-packages (11.2.1)
Requirement already satisfied: piexif in /home/luffy/.local/lib/python3.10/site-packages (1.1.3)
At the Sabaody Archipelago, Roger's first mate, Silvers Rayleigh, mentioned to Robin during the Straw Hats' meeting with him that

El texto descifrado también se ha guardado en: decrypted.txt
Texto descifrado guardado en: decrypted.txt
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ cat decrypted.txt
At the Sabaody Archipelago, Roger's first mate, Silvers Rayleigh, mentioned to Robin during the Straw Hats' meeting with him thatluffy@6
luffy@6d07a1328137:~/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Mr._0/poneglyph$ █

```

Obteniendo así, el párrafo desencriptado.

At the Sabaody Archipelago, Roger's first mate, Silvers Rayleigh, mentioned to

